

12.5.3. dlog调试

dlog是一种将格式化字符串的解析转移到主机端，其本身仅输出参数值的功能，最终效果跟串口log类似。该功能通过提取格式化字符串并从程序中分离，达到减少格式化字符串对.rodata段的占用（即减少flash的占用）

12.5.3.1. dlog的特性

1. 不保存格式化字符串，仅输出参数的值，大大减少了格式化字符串对flash的占用
2. 最大支持14个可变参数，最大支持65536句打印（可完全满足项目需求）
3. 由于仅输出参数值，所以具有更快的输出速度
4. 需要使用特定上位机解析log数据
5. 使能dlog输出到flash保存时，需要额外的外置flash和占用更多的RAM缓存
6. 与串口调试方式兼容，通过printf、puts、put_buf、putchar、log_xxx类接口输出打印信息

12.5.3.2. dlog的参数配置 🔗

1. 相关的宏配置，dlog跟串口log功能互斥，文件位置 `apps/xxx/include/app_config.h`，只能二选一,如下列举了配置频率较高的宏

```
1  //*****  
2  //          异常记录/dlog配置          //  
3  //*****  
4  #if !TCFG_DEBUG_UART_ENABLE  
5  #define TCFG_DEBUG_DLOG_ENABLE          1          // dlog功能使能  
6  #define TCFG_DEBUG_DLOG_FLASH_SEL      1          // 选择log保存到内置flash还是  
    外置flash; 0:内置flash; 1:外置flash(推荐)  
7  #define TCFG_DLOG_FLASH_START_ADDR    (0x00)      // 配置外置flash用于存储dlog  
    和异常数据的区域起始地址  
8  #define TCFG_DLOG_FLASH_REGION_SIZE   (512 * 1024) // 配置外置flash用于存储dlog  
    和异常数据的区域大小  
9  
10 #define TCFG_NORFLASH_START_ADDR      (0x00)      // 配置外置flash起始地址  
11 #define TCFG_NORFLASH_SIZE            (512 * 1024) // 配置外置flash大小  
12 #define TCFG_DEBUG_DLOG_AUTO_FLUSH_TIMEOUT (30)    // 主动刷新的超时时间(当指定  
    时间没有刷新过缓存数据到flash, 则主动刷新)(单位秒)  
13 #define TCFG_DEBUG_DLOG_UART_TX_PIN   IO_PORT_DP  // dlog串口打印的引脚  
14 #endif
```

❗ Note

1. 注意 TCFG_DLOG_FLASH_START_ADDR 和 TCFG_NORFLASH_START_ADDR 区别
 2. TCFG_DEBUG_DLOG_UART_TX_PIN 可以配置为任意引脚
 3. 不建议使用内置flash保存dlog数据
2. 相关的常量配置，文件位置 `apps/xxx/log_config/lib_system_config.c`，如下列举了配置频率较高的常量

```
1  const int config_dlog_cache_buf_num = 3;    // 配置dlog缓存的个数(4K一个), 仅  
    config_dlog_flash_enable 使能时生效  
2  const int config_dlog_flash_enable = 1;    // 使能dlog输出到flash  
3  const int config_dlog_uart_enable = 1;      // 使能dlog输出到uart
```

12.5.3.3. dlog使用步骤

1. 关闭TCFG_DEBUG_UART_ENABLE并使能TCFG_DEBUG_DLOG_ENABLE即可打开dlog功能
2. 根据实际情况进行 [dlog的参数配置](#)
3. 编译下载程序到样机中，并接上串口线
4. 打开命令行执行dlog.exe工具命令，通常位于 `cpu/xxx/tools/dlog.exe`
5. 样机上电即可观察到log输出到终端显示

❗ Note

上述步骤主要列举了实时dlog的使用步骤，如果是离线dlog，只需要解析导出的flash_log.bin即可

12.5.3.4. dlog数据的导出

对于存储在外置flash中的dlog数据，目前SDK未实现默认的dlog数据导出功能，需要由 **应用层自行开发**，可以通过蓝牙、串口等方式导出dlog数据。使用 `dlog_read_log_data` 接口可以读取dlog数据（可参考 [代码Demo](#)）

12.5.3.5. dlog数据的解析

dlog数据需要使用dlog.exe上位机并配合dlog.bin进行解析，dlog.exe解析的数据来源有两种

1. [离线log数据解析](#)：dlog数据可以保存到flash中，然后导出并保存为flash_log.bin文件，再使用dlog.exe工具解析
2. [实时log数据解析](#)：dlog数据可以通过串口实时传输到电脑，并由dlog.exe工具实时解析输出到终端显示（类似于串口log）

❗ Warning

dlog.bin 由SDK编译生成，与sdk.elf类似，每次编译都会生成不同的dlog.bin，dlog.bin必须与样机内部的程序一致，否则无法正确解析dlog数据 dlog.bin 文件在SDK的位置

`cpu/xxx/tools/dlog.bin`

12.5.3.5.1. dlog工具的使用

导出的dlog数据已经过编码，非明文log，需要通过dlog.exe工具解析后才能生成明文log，以下是dlog.exe工具的参数

```
-v 输出该工具的软件版本号
-p 指定串口号（仅用于串口实时log解析）
-b 指定串口波特率（仅用于串口实时log解析）
-d 指定串口的数据位（默认 8bit）（仅用于串口实时log解析）
-r 指定串口的校验位（默认 None）（仅用于串口实时log解析）
-s 指定串口的停止位（默认 1bit）（仅用于串口实时log解析）
-f 是否启用串口流控（默认 None）（仅用于串口实时log解析）
-i 指定dlog.bin文件的路径及文件名
-o 将解析的log输出到指定的文件
--flash 指定flash_log.bin文件（从样机导出的dlog数据）
--verbose 输出该工具的调试信息
--local_time 是否显示本地电脑的时间戳（仅用于串口实时log解析）
```

举例：

```
# 1. 使用 COM5 和 2Mb 波特率来接收串口 dlog 数据，并通过 dlog.bin 解析
.\dlog.exe -p COM5 -b 2000000 -i dlog.bin

# 2. 在例 1 的基础上加上了本地电脑的时间戳输出
.\dlog.exe -p COM5 -b 2000000 -i dlog.bin --local_time

# 3. 在例 2 的基础上加上了将 log 保存到指定文件
.\dlog.exe -p COM5 -b 2000000 -i dlog.bin -o output_log.txt --local_time

# 4. 解析指定的 flash_log.bin，并将 log 输出到终端
.\dlog.exe --flash flash_log.bin -i dlog.bin

# 5. 解析指定的 flash_log.bin，并将 log 输出到指定的文件
.\dlog.exe --flash flash_log.bin -i dlog.bin -o output_log.txt
```

将上述举例中的命令放在任意终端中执行即可，如CMD、PowerShell等。也可以通过将命令放到 bat 脚本，后续双击脚本运行

12.5.3.5.2. 离线log数据解析

假设从flash中导出的dlog数据保存为flash_log.bin文件，那么在dlog.exe工具目录执行如下命令即可解析并保存明文log打印到output_log.txt文件中

```
.\dlog.exe --flash xxx\xxx\flash_log.bin -i xxx\xxx\dlog.bin -o output_log.txt
```

12.5.3.5.3. 实时log数据解析

假设串口号为 COM5，波特率为 2M，其它参数默认，那么在dlog.exe工具目录执行如下命令即可实现解析显示明文log打印，同时保存log到output_log.txt文件中

```
.\dlog.exe -p COM5 -b 2000000 -i xxx\xxx\dlog.bin -o output_log.txt --local_time
```

⚠ Warning

由于每次编译都会生成新的dlog.bin，所以每次编译下载新程序后，需要重新打开dlog工具，已更新dlog.bin来解析新的串口打印数据

12.5.3.6. 代码Demo及log示例

12.5.3.6.1. 代码Demo

代码Demo位于文件 `apps/common/debug/dlog_config.c`

```

1 // 以下是部分离线Log接口的使用示例
2 void dlog_demo(void)
3 {
4     int ret;
5     // 1,刷新将离线Log数据从ram保存到flash
6     // 仅仅只发送一个刷新消息后退出，可以用于中断和任务
7     ret = dlog_flush2flash(0);
8     // 一直等待刷新成功，若返回成功，从函数返回后已经刷新到flash，仅可用于任务
9     ret = dlog_flush2flash(-1);
10    // 等待1000ms，若返回成功，从函数返回后已经刷新到flash，仅可用于任务
11    ret = dlog_flush2flash(100);
12
13
14    // 2,设置log的等级，主要用于 Log_xxx 类接口
15    // 设置为 info 等级，更多等级见 debug.h
16    dlog_level_set(LOG_INFO);
17
18
19    // 3,设置log输出的方式
20    // 设置为仅串口输出离线log
21    if(dlog_output_type_get() & DLOG_OUTPUT_2_FLASH){
22        // 如果之前有使能flash输出，那么关闭flash输出前需要刷新ram的log到flash
23        ret = dlog_flush2flash(100);
24    }
25    dlog_output_type_set(DLOG_OUTPUT_2_UART);
26    // 离线log输出加上flash输出(如原本是仅输出到串口,设置后log同时输出到串口和flash)
27    dlog_output_type_set(dlog_output_type_get() | DLOG_OUTPUT_2_FLASH);
28
29
30    // 4,离线log的时间、日期、序号进行同步(当收到手机下发的日期、时间后需要同步一次，以校准
    离线Log时间)
31    dlog_info_sync();
32
33
34    // 5,读取离线log数据
35    #define TMP_BUF_SIZE      501
36    u8 *tmp_buf = malloc(TMP_BUF_SIZE);
37    u32 offset = 0;
38    int ret;
39    u8 flash_log_en = (dlog_output_type_get() & DLOG_OUTPUT_2_FLASH); // 获取是否有使
    能log输出到flash
40    if(flash_log_en){
41        dlog_output_type_set(dlog_output_type_get() & (~DLOG_OUTPUT_2_FLASH)); // 禁
    止log输出到flash
42    }
43    while (1) {
44        // 建议使用dlog_read_log_data接口，而不是dlog_read_from_flash接口
45        ret = dlog_read_log_data(tmp_buf, TMP_BUF_SIZE, offset); // 读取flash中的log,
    返回0表示已读取完flash的全部log
46        if (0 == ret) {
47            // 已经全部读取
48            break;
49        }
50        offset += ret;
51        put_buf(tmp_buf, ret > 16 ? 16 : ret); // 将读取的log数据前16Byte打印出来
52    }
53    if(flash_log_en){
54        dlog_output_type_set(dlog_output_type_get() | DLOG_OUTPUT_2_FLASH); // 恢复
    log输出到flash
55    }
56    free(tmp_buf);
57 }
58

```

```

59 // 重写弱函数的实现示例
60 _WEAK_ int dlog_get_rtc_time(struct sys_time *time)
61 {
62     // 仅需返回年月日
63     u32 year = get_sys_year(); // get_sys_year函数需要自行实现,此处仅示例
64     u32 month = get_sys_month(); // get_sys_month函数需要自行实现,此处仅示例
65     u32 day = get_sys_day(); // get_sys_day函数需要自行实现,此处仅示例
66     time->year = year;
67     time->month = month;
68     time->day = day;
69
70     return 0; // 返回大于等于0表示成功
71 }
72
73 // 重写弱函数的实现示例
74 _WEAK_ u32 dlog_get_rtc_time_ms(void)
75 {
76     // dLog会多次调用这个接口获取系统时间戳,需要应用层维护一个全局时间戳,要处理好网络时间
    和本地时间的同步
77     // 返回的时间单位是毫秒,24小时制
78     //如当前时间为 20:13:30.100, 则返回 ((21 * 60 + 13) * 60) + 30) * 1000 + 100 =
    76410100毫秒
79     u32 time_ms = get_sys_time_ms(); // get_sys_time_ms函数需要自行实现,此处仅示例
80     return time_ms;
81 }

```


12.5.3.6.2. log示例

dlog实时log界面如下图所示

The screenshot shows a terminal window with the following content:

```
Active code page: 65001
D:\Desktop_File\离线log\dlog上位机\dlog_20250427.1>. \dlog.exe -p COM3 -b 2000000 --local_time -i F:\SVN_SDK\veiwable_tools\standard\TWS耳机\JL709N\download_V3.0.0\dlog.bin -o output_log.txt --verbose
[ F:\SVN_SDK\veiwable_tools\standard\TWS耳机\JL709N\download_V3.0.0\dlog.bin ]
"DLOG V1.1.2" 代码Demo及log示例

[DLOG INFO]16 代码Demo
> Dlog hash : 42044
> Dlog version : "DLOG V1.1.2"
> Log index start : 256
> Log index end : 167488
> Log addr start : 167488
> Log addr end : 520287
> Flash addr : 0 ~ 4294967295
> Exception addr : 0 ~ 0
> Timestamp : YES
> Log order : YES
> Log level : YES
> Put buffer : YES
> Put char : YES
> Time sync : YES
> Date sync : YES
> Index sync : YES
> BeiJing Time : YES

> Save file : output_log.txt
> Port : COM3
> Baudrate : 2000000
> DataBits : 8
> Parity : none
> StopBit : 1
> FlowControl : none

00000:[09:31:49.676][0000-00-00 00:00:00.874][ INFO ][WKUP]===P3_PINR_CON: 0x33
00001:[09:31:49.677][0000-00-00 00:00:00.874][ INFO ][WKUP]P3_PINR_SAFE: 0x0
00002:[09:31:49.677][0000-00-00 00:00:00.874][ INFO ][WKUP]P3_PORT_SELO: 0xa
00003:[09:31:49.677][0000-00-00 00:00:00.874][ INFO ][WKUP]P3_PINR_CON1: 0x37
```

Annotations in the image:

- Red box around the configuration section: 使能 --verbose 参数后会打印红框调试信息，非log内容
- Red box around the log output section: 当有log输入时，终端会实时解析并显示样机log

dlog生成的明文log如下图所示，图中从左到右红框依次表示：log序号、手机/电脑时间戳、日期、耳机本地时间戳、log等级

The screenshot shows a text file with the following log entries:

```
31 00030:[09:31:49.694][0000-00-00 00:00:00.892][ INFO ][FRE_OFFSET]
32 --func=bt fre offset storage init
33 00031:[09:31:49.694][0000-00-00 00:00:00.892][ INFO ][APP-UPDATE]<-----update_result_deal=0x5a00 628d-----
34 00032:[09:31:49.694][0000-00-00 00:00:00.892][UNKNOW]vbat: 4400
35 00033:[09:31:49.694][0000-00-00 00:00:00.892][UNKNOW]vbat: 4400
36 00034:[09:31:49.694][0000-00-00 00:00:00.892][UNKNOW]vbat: 4400
37 00035:[09:31:49.694][0000-00-00 00:00:00.892][UNKNOW]vbat: 4400
38 00036:[09:31:49.694][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
39 00037:[09:31:49.694][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
40 00038:[09:31:49.694][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
41 00039:[09:31:49.694][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
42 00040:[09:31:49.696][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
43 00041:[09:31:49.696][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
44 00042:[09:31:49.696][0000-00-00 00:00:00.893][UNKNOW]vbat: 4400
45 00043:[09:31:49.696][0000-00-00 00:00:00.893][ INFO ][APP_POWER_ON]power on
46 00044:[09:31:49.696][0000-00-00 00:00:00.894][UNKNOW]tone_player: tone_en/power_on.*
47 00045:[09:31:49.696][0000-00-00 00:00:00.894][ INFO ][BSC][3lmBSC][lmpipeline_uuid: 7674
48 BSC[0m
49 00046:[09:31:49.697][0000-00-00 00:00:00.894][ DEBUG][CLOCK]---sys clk set : 192000000
50 00047:[09:31:49.697][0000-00-00 00:00:00.894][UNKNOW]clk_div 4, lsb_clk:24000000
```

Annotations in the image:

- Red boxes around the log entries: log序号、手机/电脑时间戳、日期、耳机本地时间戳、log等级

❗ Note

1. log序号表示耳机开机到输出当前log时，一共输出了多少条log
2. 日期为全0是由于日期需要耳机实现 dlog_get_rtc_time 接口，默认没有实现
3. 如果是离线log，还需要实现 dlog_get_rtc_time_ms 接口，否则手机/电脑时间戳栏也为耳机本地时间戳

log 等级定义如下：

等级	接口
VERB	log_v、log_verb
DEBUG	log_d、log_debug
INFO	log_i、log_info
WARN	log_w、log_warn
ERROR	log_e、log_error
CHAR	log_c、log_char
UNKNOWN	printf、put_buf、puts

12.5.3.7. 自定义log保存方式

SDK默认已经支持log保存到内置、外置flash，代码位于 `apps/common/debug/dlog_config.c`，如果应用层需要实现更多的log保存方式，可参考并实现如下接口

```

1  /*-----*/
2  /**@brief 获取保存dLog数据的flash起始地址和大小
3  @param addr: 返回起始地址
4  @param len: 返回长度
5  @param offset: 数据需要写入的flash地址
6  @return 等于0表示成功; 小于0表示失败
7  @note 起始地址和长度必需 4K 对齐
8  */
9  /*-----*/
10 static int dlog_get_xxx_zone(u32 *addr, u32 *len)
11
12 /*-----*/
13 /**@brief 将flash的指定扇区擦除
14 @param erase_sector: 要擦除的扇区偏移(偏移是相对于保存dLog数据的flash区域起始地址)
15 @param sector_num: 要擦除的扇区个数
16 @param offset: 数据需要写入的flash地址
17 @return 等于0表示成功; 小于0表示失败
18 @note
19 */
20 /*-----*/
21 static int dlog_xxx_zone_erase(u16 erase_sector, u16 sector_num)
22
23 /*-----*/
24 /**@brief 将dLog数据写入flash
25 @param buf: 要写入的数据
26 @param len: 要写入的数据长度
27 @param offset: 数据需要写入的flash地址
28 @return 返回写入的长度, 返回值小于0表示写入失败
29 @note
30 */
31 /*-----*/
32 static int dlog_xxx_write(void *buf, u16 len, u32 offset)
33
34 /*-----*/
35 /**@brief 从flash中读dLog数据
36 @param buf: 返回要读取的数据
37 @param len: 需要读取的数据长度
38 @param offset: 需要读取的flash地址
39 @return 返回写入的长度, 返回值小于0表示写入失败
40 @note
41 */
42 /*-----*/
43 static int dlog_xxx_read(void *buf, u16 len, u32 offset)
44
45 // 返回 0 表示初始化成功
46 static int dlog_xxx_init(void)
47
48 // 实现的接口需要通过如下方式注册给dLog模块使用
49 REGISTER_DLOG_OPS(xxx_op, 1) = {
50     .dlog_output_init      = dlog_xxx_init,
51     .dlog_output_get_zone  = dlog_get_xxx_zone,
52     .dlog_output_zone_erase = dlog_xxx_zone_erase,
53     .dlog_output_write     = dlog_xxx_write,
54     .dlog_output_read      = dlog_xxx_read,
55     .dlog_output_direct    = dlog_output_direct,    // 这个接口不需要实现, 可直接引用
56 };

```

12.5.3.8. 常见问题

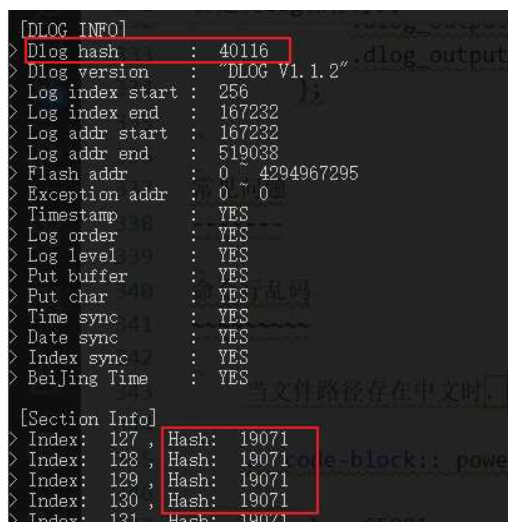
12.5.3.8.1. 命令行乱码

当文件路径存在中文时，如果命令行出现中文乱码问题，可以先执行下面命令，再执行dlog命令

```
chcp 65001
```

12.5.3.8.2. 生成的log文件大小为0

在使用dlog工具解析离线log时，生成的log文件大小为0通常是由于样机的程序与dlog.bin不匹配，通过下图打印的hash可以确认



```
[DLOG INFO]
> Dlog hash      : 40116
> Dlog version   : "DLOG V1.1.2"
> Log index start : 256
> Log index end   : 167232
> Log addr start  : 167232
> Log addr end    : 519038
> Flash addr     : 0 4294967295
> Exception addr  : 0 0
> Timestamp      : YES
> Log order       : YES
> Log level       : YES
> Put buffer      : YES
> Put char        : YES
> Time sync       : YES
> Date sync       : YES
> Index sync      : YES
> Beijing Time    : YES

[Section Info]
> Index: 127, Hash: 19071
> Index: 128, Hash: 19071
> Index: 129, Hash: 19071
> Index: 130, Hash: 19071
> Index: 131, Hash: 19071
```

12.5.3.8.3. 无法打开串口

dlog工具报 “[ERROR] Serial port open fail.”，检查串口号是否被其它软件占用

12.5.3.8.4. 无法打开文件

dlog工具报 “[ERROR] The dlog file open fail.”，请检查文件路径和文件名是否正确，是否存在中文乱码

12.5.3.8.5. 出现log丢失

该问题主要出现在离线的log中，通常出现这种问题原因有以下几点：

- 1. 打印太多的log，由于flash写入速度较慢，导出缓存写满，而丢失最新打印
- 2. 系统过于繁忙，无法及时将缓存的log刷新到flash中

解决方法：

- 1. 优化log，如提高log等级，减少不必要的打印
- 2. 加大缓存，通过配置 config_dlog_cache_buf_num 参数加大缓存

12.5.3.8.6. 实时log出现部分打印解析出错

检查dlog.bin与下载到样机的程序是否匹配，检查重新下载程序后有没有重新打开dlog工具

12.5.3.9. 注意事项

- 1. dlog工具无法识别实时串口log数据与dlog.bin是否匹配，需要用户确认，否则输出的log会有误（离线log数据解析无此问题）
- 2. 打印接口最大仅支持14个可变的参数，如果参数个数超过14个，可以分成2次进行打印

12.5.3.10. dlog的flash消耗对比

dlog 不需要保存格式化字符串到.rodata段，因此可以减少对flash的使用，对比结果如下表格所示：

...	使能dlog	使能普通串口打印	完全关闭打印	使能dlog并关闭
打印使用的flash	96KB	160KB	0KB	55KB

Note

- 1. 主要是指减少用于保存程序的内置flash消耗，用于保存dlog数据的外置flash不在讨论范围，即使没有外置flash也能通过串口输出实时的dlog打印
- 2. 该参考数据基于可视化耳机SDK测试得到
- 3. 目前断言打印占用了较多的flash空间，flash较少的项目可以关闭断言 (config_asser 设置为0)

12.5.3.11. dlog接口使用说明

dlog.h文件有详细接口说明，此处列举常用接口。文件位置 `interface/utls/generic/dlog.h`

```

1  /*-----*/
2  /**@brief dLog功能的临时使能/失能
3  @param en
4          0 : 禁止打印数据输出(禁止输出到缓冲区)
5          1 : 使能打印数据输出(使能输出到缓冲区)
6  @return void
7  @note 只是临时开关打印数据输出, 不刷新缓冲区数据到flash, (可自行调用flush接口)
8  */
9  /*-----*/
10 void dlog_enable(u8 en);
11
12 /*-----*/
13 /**@brief 更新 ram 的数据到 flash 中
14 @param timeout 刷新等待时间
15          0 : 不等待, 只发送消息给任务刷新dLog, 不管是否刷新成功, 都马上退出
16          -1: 一直等待, 直到刷新成功才退出该函数
17          xx: 刷新成功则马上退出, 否则等待 xx * 10ms的超时时间, 直到刷新成功
18 @return 成功返回 0, 失败返回非 0
19 @note 不可在中断/关中断传入非 0 参数值
20 */
21 /*-----*/
22 int dlog_flush2flash(u32 timeout);
23
24 /*-----*/
25 /**@brief 读取flash中的dLog信息
26 @param buf : 返回读取的数据
27 @param len : 需要读取的数据长度
28 @param offset : 读取数据的偏移(偏移是相对于flash区域起始地址)
29 @return 返回读取到的数据长度, 0表示已读取完数据
30 @note
31 */
32 /*-----*/
33 u16 dlog_read_from_flash(u8 *buf, u16 len, u32 offset);
34
35 /*-----*/
36 /**@brief 读取flash中的dLog和异常数据
37 @param buf : 返回读取的数据
38 @param len : 需要读取的数据长度
39 @param offset : 读取数据的偏移(偏移是相对于flash区域起始地址)
40 @return 返回读取到的数据长度, 0表示已读取完数据
41 @note
42 */
43 /*-----*/
44 u16 dlog_read_log_data(u8*buf, u16 len, u32 offset);
45
46 /*-----*/
47 /**@brief 设置dLog等级
48 @param level : 等级, 如下宏定义所示
49             __LOG_VERB      0
50             __LOG_DEBUG     1
51             __LOG_INFO      2
52             __LOG_WARN      3
53             __LOG_ERROR     4
54             __LOG_CHAR      5
55 @return void
56 @note
57 */
58 /*-----*/
59 void dlog_level_set(int level);
60
61 /*-----*/
62 /**@brief 获取dLog等级
63 @param void

```



```

64  @return level : 等级, 如下宏定义所示
65          __LOG_VERB      0
66          __LOG_DEBUG     1
67          __LOG_INFO      2
68          __LOG_WARN      3
69          __LOG_ERROR     4
70          __LOG_CHAR      5
71  @note
72  */
73  /*-----*/
74  int dlog_level_get(void);
75
76  /*-----*/
77  /**@brief 设置dlog的输出类型
78  @param type :
79          DLOG_OUTPUT_2_NONE: 既不输出到串口也不输出到flash, 仅输出到缓存
80          DLOG_OUTPUT_2_FLASH: 仅输出到flash
81          DLOG_OUTPUT_2_UART: 仅输出到串口
82  @return : 如果返回值不为0, 则表示缓存数据没有刷新到flash(非丢数据)
83  @note 需要同时输出到串口和flash可用位或(DLOG_OUTPUT_2_UART | DLOG_OUTPUT_2_FLASH)
84  */
85  /*-----*/
86  int dlog_output_type_set(enum DLOG_OUTPUT_TYPE type);
87
88  /*-----*/
89  /**@brief 获取dlog的输出类型
90  @param void
91  @return : enum DLOG_OUTPUT_TYPE
92  @note
93  */
94  /*-----*/
95  enum DLOG_OUTPUT_TYPE dlog_output_type_get(void);
96
97  /*-----*/
98  /**@brief 输出dlog的同步信息
99  @param void
100 @return : 大于等于0表示执行成功
101 @note 由于时间戳和序号采用增量形式, 所以通过该接口可以输出一个绝对值进行同步
102 */
103 /*-----*/
104 int dlog_info_sync(void);
105
106 /*-----*/
107 /**@brief dlog用于获取rtc时间的年月日, 需要外部重写实现, 若不支持rtc时间可忽略
108 @param time : 用于保存返回的年月日
109 @return : 大于等于0表示成功获取
110 @note
111 */
112 /*-----*/
113 _WEAK_ int dlog_get_rtc_time(struct sys_time *time);
114
115 /*-----*/
116 /**@brief dlog用于获取rtc时间的时分秒毫秒, 需要外部重写实现, 若不支持rtc时间可忽略
117 @param void
118 @return : 返回时分秒毫秒组成的一个整数值, 24小时制
119 @note 如当前时间为 21:13:30.100, 则返回 ((21 * 60 + 13) * 60 + 30) * 1000 + 100 =
120 76410100
121 */
122 /*-----*/
123 _WEAK_ u32 dlog_get_rtc_time_ms(void);

```

❗ Note

1. 如果需要使用rtc日期时间输出，需要用户实现 `dlog_get_rtc_time` 和 `dlog_get_rtc_time_ms` 接口（可参考 [代码Demo](#)）
2. 推荐使用 `dlog_read_log_data` 读取dlog的log数据